CS612 · ADVANCED ALGORITHMS · MSC · FALL 2026

Lecture 02
Divide-and-Conquer, Recurrences & Selection

Sorting needs $\Omega(n \log n)$. Does finding the median? And how do we read a recursive algorithm's cost off its structure?

Saturday 3 October 2026 · 3 hours

PS1 released today (due Sat 17 Oct)

Where we are

L01 established

- A complexity claim = model + input size + cost measure + guarantee type + bound
- Correctness = spec + invariant/induction + termination
- Comparison sorting needs $\lceil \log_2 n! \rceil$ comparisons; adversary and decision-tree arguments

TODAY'S QUESTION

Finding the k-th smallest of n keys: can we beat sorting's $\Theta(n \log n)$? If yes, what kind of guarantee do we get: **expected** or **worst case**?

To answer it, we need tools to analyse recursive algorithms whose recursion is *unbalanced*.

Motivating problem: percentiles at scale

A monitoring system keeps $n = 10^9$ request latencies in memory. Every minute it reports the **median**, the **p99** and the **p99.9**.

- Sorting: $\approx n \log_2 n \approx 3 \times 10^{10}$ comparisons per report.
- Is there a lower bound that makes this unavoidable?
- The p99 of 10^9 values is the 9.9×10^8-th smallest. A heap of the top 1% has 10^7 entries. Is that better?

DISCUSS · 3 MIN

What is the **best lower bound you can prove right now** for finding the median in the comparison model? Is it $\Omega(n \log n)$?

BLOCK A · 00:10–01:00

Formulating and solving recurrences

Substitution · recursion trees · Master theorem · where it breaks

From algorithm to recurrence

Algorithm	Structure	Recurrence	Solution
Binary search	1 subproblem of half size, $O(1)$ work	$T(n) = T(n/2) + \Theta(1)$	$\Theta(\log n)$
Merge sort	2 halves, linear merge	$T(n) = 2T(n/2) + \Theta(n)$	$\Theta(n \log n)$
Karatsuba multiplication	3 half-size products, $O(n)$ additions	$T(n) = 3T(n/2) + \Theta(n)$	$\Theta(n^{\log_2 3}) \approx n^{1.585}$
Strassen	7 half-size matrix products, $\Theta(n^2)$ additions	$T(n) = 7T(n/2) + \Theta(n^2)$	$\Theta(n^{\log_2 7}) \approx n^{2.807}$
Median of medians	**two *different-sized* recursive calls**	$T(n) \le T(n/5) + T(7n/10) + \Theta(n)$	**? (today)**

> **Floors and ceilings** ($T(\lceil n/2 \rceil)$ etc.) almost never change the Θ bound. The Akra–Bazzi theorem justifies ignoring them for a broad class of recurrences.

Substitution: guess, then prove by induction

Example. $T(n) = 2T(\lfloor n/2 \rfloor) + 1$. Guess $T(n) = O(n)$.

Naive attempt: assume $T(m) \le cm$ for $m < n$:

$$T(n) \le 2c\lfloor n/2 \rfloor + 1 \le cn + 1.$$

$cn + 1 \not\le cn$. The induction **fails**, even though the guess is right.

Fix: strengthen the hypothesis. Assume $T(m) \le cm - d$:

$$T(n) \le 2\big(c\lfloor n/2 \rfloor - d\big) + 1 \le cn - 2d + 1 \le cn - d$$

whenever $d \ge 1$. Base case: choose c large relative to $T(1)$.

RULE

The inductive step must prove *exactly* the hypothesis, with the same constants. "$cn + 1 = O(n)$" is **not** a valid step.

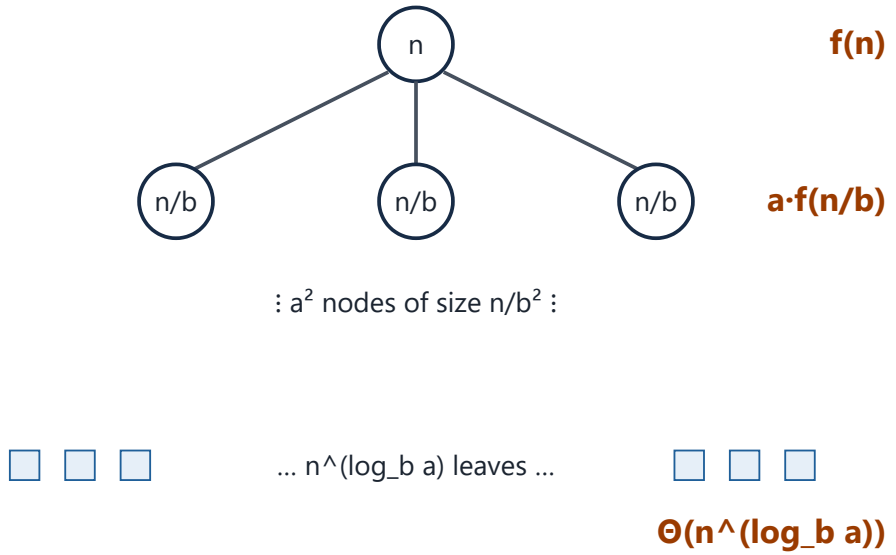
Recursion trees: where does the cost live?

$T(n) = aT(n/b) + f(n)$: level j has a^j nodes of size n/b^j, for total cost $a^j f(n/b^j)$. Depth $\log_b n$, with $a^{\log_b n} = n^{\log_b a}$ leaves.

$$T(n) = \sum_{j=0}^{\log_b n - 1} a^j f\left(\frac{n}{b^j}\right) + \Theta\left(n^{\log_b a}\right)$$

The per-level costs form (roughly) a geometric series. Three regimes:

- **decreasing** down the tree → the root dominates → $\Theta(f(n))$
- **constant** → $\Theta(f(n) \log n)$
- **increasing** → the leaves dominate → $\Theta(n^{\log_b a})$



The Master theorem, precisely

THEOREM (CLRS 4E, THM 4.1)

Let $a \geq 1$, $b > 1$, f asymptotically positive, and $T(n) = aT(n/b) + f(n)$. Let $w = \log_b a$ (the *watershed exponent*).

1. If $f(n) = O(n^{w-\varepsilon})$ for some $\varepsilon > 0$: $T(n) = \Theta(n^w)$.
2. If $f(n) = \Theta(n^w \log^k n)$ for some $k \geq 0$: $T(n) = \Theta(n^w \log^{k+1} n)$.
3. If $f(n) = \Omega(n^{w+\varepsilon})$ for some $\varepsilon > 0$ **and** $af(n/b) \leq cf(n)$ for some $c < 1$ and all large n (regularity): $T(n) = \Theta(f(n))$.

The "basic" textbook version has only $k = 0$ in case 2. The $\log^k$ extension covers $2T(n/2) + n \log n = \Theta(n \log^2 n)$.

Cases 1 and 3 need a **polynomial** gap n^{ε}. A log-factor gap is not enough.

Where the Master theorem does not apply

Recurrence	Why not	Tool	Answer
$4T(n/2) + n^2/\log n$	f is below n^2 by only a log factor (and $k = -1 < 0$)	recursion tree	$\Theta(n^2 \log\log n)$
$T(n/3) + T(2n/3) + n$	unequal subproblem sizes	tree + substitution	$\Theta(n \log n)$
$T(n/5) + T(7n/10) + n$	unequal sizes, fractions sum to 0.9	substitution	$\Theta(n)$
$\sqrt{n}\,T(\sqrt{n}) + n$	a is not constant	divide by n; change variables	$\Theta(n \log\log n)$
$T(n-1) + n$	subtractive, not divisive	unroll / sum	$\Theta(n^2)$

GENERAL TOOL (FOR REFERENCE)

Akra–Bazzi: $T(n) = \sum_i a_i T(b_i n) + g(n)$. Find p with $\sum a_i b_i^p = 1$; then $T(n) = \Theta\big(n^p(1 + \int_1^n \frac{g(u)}{u^{p+1}}du)\big)$.

Exercise A: classify and solve

1. $T(n) = 8T(n/2) + n^2$
2. $T(n) = 2T(n/4) + \sqrt{n}$
3. $T(n) = 2T(n/2) + n/\log n$
4. $T(n) = 3T(n/3) + n \log^2 n$
5. $T(n) = T(n/2) + T(n/4) + n$
6. $T(n) = 2T(n/2) + n^2$

PAIRS · 7 MIN

For each: does the Master theorem apply? Which case (and is regularity satisfied)? If not, which tool would you use? Give the Θ bound.

BLOCK B · 01:10–02:00

Case studies: when D&C wins, and when it doesn't

Maximum subarray · Strassen · selection in linear time

Maximum subarray: D&C, then do better

D&C: $\Theta(n \log n)$

The best subarray lies entirely in the left half, entirely in the right half, or **crosses** the midpoint. The crossing case = the best suffix of the left half + the best prefix of the right half: $\Theta(n)$.

$T(n) = 2T(n/2) + \Theta(n) = \Theta(n \log n)$.

But: Kadane's scan, $\Theta(n)$

```
1 KADANE(A)
2   best = A[1] ; cur = A[1]
3   for i = 2 to n
4       cur = max(A[i], cur + A[i])
5       best = max(best, cur)
6   return best
```

INVARIANT (AFTER STEP I)

cur = the maximum sum of a subarray **ending at** i; $best$ = the maximum sum of any subarray within $A[1..i]$.

LESSON

D&C gave a correct $n \log n$ algorithm, but a **stronger invariant** (a DP state, as in L06) gives n. The recurrence tells you what an algorithm costs; it does not tell you whether a better algorithm exists.

Strassen: trading multiplications for additions

Split $n \times n$ matrices into 2×2 blocks of size $n/2$. The standard method needs 8 block products → $T = 8T(n/2) + \Theta(n^2) = \Theta(n^3)$.

Strassen (1969): **7** block products $P_1 .. P_7$ plus 18 block additions/subtractions:

$$T(n) = 7T(n/2) + \Theta(n^2) \;\Rightarrow\; \Theta(n^{\log_2 7}) = \Theta(n^{2.807\ldots})$$

Case 1 of the Master theorem: $n^2 = O(n^{2.807-\varepsilon})$.

THEORY VS. PRACTICE

- Crossover with tuned $\Theta(n^3)$ BLAS: typically n in the hundreds to low thousands (it depends on the machine).
- Weaker numerical stability (subtractions amplify floating-point error).
- Extra memory for temporaries.
- The best known exponent today is $\omega < 2.372$ (2024), but those algorithms are "galactic".

Selection: specification and a first algorithm

SELECT(A, k)**:** given n distinct keys and $1 \le k \le n$, return the key of rank k (the k-th smallest).

```
1 RAND-SELECT(A, k)
2   if |A| == 1 return A[1]
3   p = uniformly random element of A
4   L = {x ∈ A : x < p} ; R = {x ∈ A : x > p}
5   if k ≤ |L|      return RAND-SELECT(L, k)
6   if k == |L| + 1 return p
7   return RAND-SELECT(R, k − |L| − 1)
```

Correctness by strong induction on $|A|$: p has rank $|L| + 1$, and the rank of the answer within L or R is adjusted accordingly.

EXPECTED $O(n)$, INTUITION

With probability $\ge 1/2$ the pivot's rank lies in the middle half, and then the recursion keeps at most $3n/4$ elements. On average, 2 partitions are needed to shrink by $3/4$:

$$E[T(n)] \le E[T(3n/4)] + 2cn \;\Rightarrow\; O(n).$$

Full indicator-variable analysis: L03.

Worst case: $\Theta(n^2)$ (the pivot is always an extreme).

Median of medians (Blum, Floyd, Pratt, Rivest, Tarjan 1973)

```
SELECT(A, k)                    ▷ deterministic, n = |A|
  if n ≤ 140  sort A and return A[k]
  split A into ⌈n/5⌉ groups of ≤ 5 elements
  M = the median of each group        ▷ O(n): ≤ 6 comparisons per group
  x = SELECT(M, ⌈|M|/2⌉)              ▷ recursive: median of medians
  partition A around x into L, {x}, R
  if k ≤ |L|      return SELECT(L, k)
  if k == |L| + 1 return x
  return SELECT(R, k − |L| − 1)
```

TWO RECURSIVE CALLS

one on the $\lceil n/5 \rceil$ medians (to *find* a good pivot), one on the side that contains the answer (to *use* the pivot).

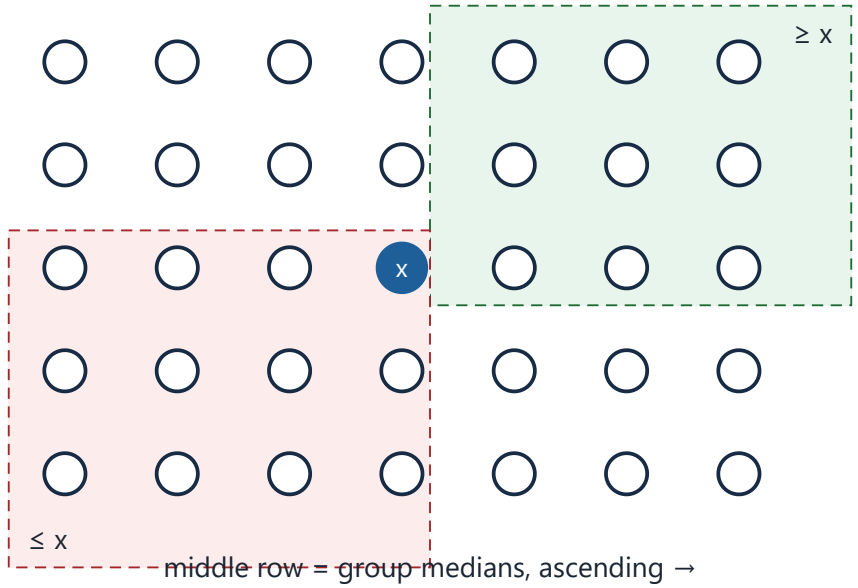
Key question: how large can L or R be?

The key lemma: the pivot is at least 30% in

Arrange the groups as columns, each sorted with its **largest element at the top**, and order the columns left to right by increasing median. The pivot x is the median of the middle row.

Lemma. At least $\frac{3n}{10} - 6$ elements are $> x$, and likewise $< x$.

Proof. At least half of the $\lceil n/5 \rceil$ groups have a median $\geq x$. Each such group, except possibly x's own group and a final partial group, contributes 3 elements $\geq x$ (its median and the two below it). So there are at least $3\left(\frac{1}{2}\lceil \frac{n}{5} \rceil - 2\right) \geq \frac{3n}{10} - 6$ such elements. Hence $|L|, |R| \leq \frac{7n}{10} + 6$.



Linear time: the substitution proof

$T(n) \le T(\lceil n/5 \rceil) + T(\frac{7n}{10} + 6) + an$ for $n > 140$, with $T(n) = O(1)$ for $n \le 140$.

CLAIM: $T(n) \le cn$ FOR A SUITABLE c

Assume it holds for all smaller arguments. Then

$$T(n) \le c\left(\tfrac{n}{5} + 1\right) + c\left(\tfrac{7n}{10} + 6\right) + an = \tfrac{9}{10}cn + 7c + an.$$

This is $\le cn$ iff $c\left(\tfrac{n}{10} - 7\right) \ge an$. For $n \ge 140$, $\tfrac{n}{10} - 7 \ge \tfrac{n}{20}$, so $c \ge 20a$ suffices (and also choose c large enough for the base cases).

■

WHY IT WORKS

$\frac{1}{5} + \frac{7}{10} = \frac{9}{10} < 1$: the total work shrinks geometrically, level by level.

THE PRICE

$c \ge 20a$: a large constant. In our experiment, about **8.4n** comparisons vs. **3.4n** for random pivots.

Why group size 5? (And why not 3?)

For odd group size g, about half the groups contribute $\frac{g+1}{2}$ elements on each side of x:

$$\text{discarded} \gtrsim \frac{g+1}{4g}\,n, \qquad |L|, |R| \lesssim \frac{3g-1}{4g}\,n.$$

Sizes of the two recursive calls: $\frac{n}{g} + \frac{3g-1}{4g}n = \frac{3g+3}{4g}\,n < n \iff g > 3$.

g	Recurrence	Result
3	$T(n/3) + T(2n/3) + O(n)$	$\Theta(n \log n)$
5	$T(n/5) + T(7n/10) + O(n)$	$\Theta(n)$
7	$T(n/7) + T(5n/7) + O(n)$	$\Theta(n)$

THINK · 2 MIN

Larger groups make the fractions smaller, but sorting each group costs more. What does this trade-off look like? Is group size 3 *actually* slow in practice? (See Block C.)

BLOCK C · 02:10–02:55

Choosing a selection method

Guarantees · measurements · scenarios

Four ways to find the k-th smallest

Method	Time	Guarantee type	Extra space	Robust to adversarial input?
Sort, then index	$\Theta(n \log n)$	worst case	$O(1)$–$O(n)$	yes; answers *all* ranks afterwards
Heap: build + k extract-min	$\Theta(n + k \log n)$	worst case	$O(1)$ in place	yes
Size-k heap over a stream	$\Theta(n \log k)$	worst case	$\Theta(k)$	yes; works on streams
Randomized quickselect	$\Theta(n)$	**expected** ($\Theta(n^2)$ worst)	$O(1)$ iterative	yes, *if* the adversary cannot predict the pivots
Median of medians	$\Theta(n)$	worst case	$O(\log n)$ stack	yes, but with a large constant
Introselect (`std::nth_element`)	**$\Theta(n)$ typical**	**worst case $O(n \log n)$ (libstdc++ falls back to heapselect)**	$O(1)$	**yes**

Experiment: median selection, measured

n	input	QS cmp/n	QS ms	MoM-5 cmp/n	MoM-5 ms	MoM-3 cmp/n	MoM-3 ms	sort cmp/n	sort ms	nth_element ms
10^5	random	2.94	0.7	8.36	3.7	10.05	3.9	20.60	6.9	0.8
10^6	random	3.59	8.2	8.42	37.5	10.61	40.8	23.53	77.2	7.7
10^7	random	3.40	97.0	8.44	391.1	10.96	433.0	28.04	927.0	82.7
10^7	sorted	3.72	46.6	7.29	141.8	7.88	179.1	31.65	532.5	18.3

Distinct 64-bit keys, $k = n/2$; comparisons counted; time = median of 5 runs; g++ 15.2 -O2. `Student_Resources/selection_bench.cpp`.

INTERPRET · 4 MIN

(1) Which column confirms a theorem, and which theorem? (2) MoM-3 is $\Theta(n \log n)$ in the worst case. Does the data show it? (3) Why is everything faster on sorted input?

Exercise C: pick the method (and defend it)

1. A public web API returns the median of a client-supplied array ($n \le 10^6$) under a strict latency SLO. Attackers may submit crafted arrays.
2. A stream of 10^{10} events; report the **100 largest** values at the end. Memory: 1 GB.
3. A fixed data set of $n = 10^8$ values will receive 10^5 rank queries with different k over the next hour.
4. An in-memory array of $n = 10^9$ values; one report of the 1st, 2nd, …, 99th percentiles.

> **GROUPS · 12 MIN**
>
> For each: the method, its cost with the guarantee type, and the **assumption** that, if it changed, would change your answer.

Synthesis

1. Write the recurrence from the algorithm: number of subproblems, their sizes, non-recursive work.
2. Recursion tree to **guess**; substitution to **prove**, with the exact hypothesis (strengthen if needed).
3. Master theorem: compare $f(n)$ with $n^{\log_b a}$; needs a polynomial gap (cases 1/3) plus regularity (case 3). Know the gaps and the unequal-split cases.
4. Selection is $\Theta(n)$, strictly easier than sorting: expected via random pivots, worst case via median of medians ($\frac{1}{5} + \frac{7}{10} < 1$).
5. Choosing a method depends on the guarantee type, the constants, the adversary, the memory, and the number of queries.

NEXT WEEK · L03 (SAT 10 OCT)

Randomized algorithms properly: Las Vegas vs. Monte Carlo, expected time vs. failure probability, indicator-variable analysis of quicksort and quickselect, amplification.

Read: CLRS 4e Ch. 5, 7.3–7.4, 9.2. **PS1** released today, due Sat 17 Oct. **Project specification** next week.